Day 154 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 1 reference

Watch: Collaborative Filtering : Data Science Concepts (ritvikmath) | Recommender Systems 4 Item Item Collaborative Filtering (From Languages to Information)

`day154_collaborative_filtering.py`, segment by segment, in the order `main()` uses things. The parts (data loading, the split, the ratings table, the bias model, the similarity and the two neighbour predictors) are copied unchanged into `day154_exercise.py` for the week's build. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: R = ratings table (users x films) · M = where it is filled · μ, b_u, b_i = mean and biases · S = similarities · n = shared raters · k = neighbours · D = deviations or residuals

Segment 0 — Setup

Imports and settings

CODE

```
import os
import sys
import time
import zipfile
import urllib.request
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
HERE = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(os.path.expanduser("~"), "movielens_data")
URL = "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
KS = (5, 10, 20, 40, 80, 160)
SHRINKS = (0, 10, 25, 50, 100, 200, 400, 800)
LAMS = (0, 2, 5, 10, 25, 50)
DAMPS = (0, 0.1, 0.25, 0.5, 1, 2, 4)
```

- `import urllib.request` — the standard library's downloader: no extra package is needed to fetch the data.
- `os.path.expanduser("~")` — the home folder (C:\Users\name on Windows), so the data lives outside the repository.
- `sys.stdout.reconfigure(encoding="utf-8")` — film titles contain accents; this stops a print from failing on an old console.
- `SHRINKS = (0, 10, ..., 800)` — a tuple of the settings tried; tuples cannot be changed by accident.

Segment 1 — The parts

load_movielens and load_items: the data

CODE

```
def load_movielens(folder=DATA):
    path = os.path.join(folder, "ml-100k", "u.data")
    if not os.path.exists(path):
        os.makedirs(folder, exist_ok=True)
        z = os.path.join(folder, "ml-100k.zip")
        urllib.request.urlretrieve(URL, z)
        zipfile.ZipFile(z).extractall(folder)
    a = np.loadtxt(path, dtype=np.int64)
    return a[:, 0] - 1, a[:, 1] - 1, a[:, 2].astype(float), a[:, 3]
def load_items(folder=DATA):
    rows = [ln.split("|") for ln in open(os.path.join(folder, "ml-100k", "u.item"),
        encoding="latin-1").read().strip().split("\n")]
    names = [ln.split("|")[0] for ln in open(os.path.join(folder, "ml-100k", "u.genre"),
        encoding="latin-1").read().strip().split("\n")]
    titles = [r[1] for r in rows]
    genres = np.array([[int(g) for g in r[5:24]] for r in rows])
    return titles, genres, names
```

- `if not os.path.exists(path):` — download only the first time; afterwards the file is read from disk.
- `zipfile.ZipFile(z).extractall(folder)` — unpacks every file in the archive into the folder.
- `np.loadtxt(path, dtype=np.int64)` — reads the tab-separated numbers into a 100,000 x 4 table of whole numbers.
- `a[:, 0] - 1` — MovieLens counts users and films from 1; Python indexes from 0.
- `encoding="latin-1"` — the file is not UTF-8; reading it as latin-1 keeps titles such as 'Belle de jour' intact.
- `r[5:24]` — a slice of fields 5 to 23: the 19 genre flags.

split_ratings and to_matrix

CODE

```
def split_ratings(n, frac, seed):
    p = np.random.RandomState(seed).permutation(n)
    m = int(round(n * frac))
    return np.sort(p[m:]), np.sort(p[:m])
def to_matrix(u, i, r, nu, ni):
    R = np.zeros((nu, ni)); M = np.zeros((nu, ni), bool)
    R[u, i] = r; M[u, i] = True
    return R, M
```

- `np.random.RandomState(seed).permutation(n)` — 0 to n – 1 in a random order that depends only on the seed.
- `np.sort(p[m:]), np.sort(p[:m])` — a function can return two arrays as one tuple; sorting keeps file order.
- `R[u, i] = r` — fancy indexing: puts each rating in its (user, film) cell in one statement.

fit_biases: two alternating exact steps

MATH

$$b_i = \Sigma_u (r_{ui} - \mu - b_u) / (n_i + \lambda_i); \quad b_u = \Sigma_i (r_{ui} - \mu - b_i) / (n_u + \lambda_u)$$

CODE

```
def fit_biases(R, M, lam_u, lam_i, iters=20):
    mu = R[M].mean()
    bu = np.zeros(R.shape[0]); bi = np.zeros(R.shape[1])
    nu, ni = M.sum(1), M.sum(0)
    for _ in range(iters):
        bi = ((R - mu - bu[:, None]) * M).sum(0) / np.maximum(ni + lam_i, 1e-9)
        bu = ((R - mu - bi[None, :]) * M).sum(1) / np.maximum(nu + lam_u, 1e-9)
    return mu, bu, bi
```

- `R[M].mean()` — a boolean mask picks the filled cells; their mean is μ.
- `bu[:, None]` — a column (users x 1), so it broadcasts across every film.
- `(...) * M` — multiplying by the mask zeroes the empty cells, so sums run over real ratings only.
- `np.maximum(ni + lam_i, 1e-9)` — guards against 0 / 0 for a film nobody rated when the penalty is 0.

similarity: every pair at once

MATH

$$sim(a, b) = \Sigma_u x_{ua} x_{ub} / (\sqrt{\Sigma_u x_{ua}^2 m_{ub}} \cdot \sqrt{\Sigma_u m_{ua} x_{ub}^2}); \quad shrunk: \times n / (n + shrink)$$

CODE

```
def similarity(X, M, shrink=0.0):
    Mf = M.astype(float); X2 = X * X
    num = X.T @ X
    den = np.sqrt(X2.T @ Mf) * np.sqrt(Mf.T @ X2)
    S = np.divide(num, den, out=np.zeros_like(num), where=den > 0)
    n = Mf.T @ Mf
    if shrink:
        S = S * n / (n + shrink)
    np.fill_diagonal(S, 0.0)
    return S, n
```

- `X.T @ X` — a (films x films) table of dot products; empty cells are 0, so only shared raters add anything.
- `X2.T @ Mf` — for each pair (a, b): the sum of a's squared values over the users who also rated b.
- `np.divide(num, den, out=..., where=den > 0)` — divides only where it is safe; elsewhere the 'out' value (0) stays.
- `np.fill_diagonal(S, 0.0)` — sets S[a, a] = 0 in place, so a film is never its own neighbour.

_groups and _weighted: the shared machinery

MATH

$$correction = \Sigma_{top\ k,\ s > 0}\ s \cdot d\ /\ (\Sigma\ s + damp)$$

CODE

```
def _groups(keys):
    order = np.argsort(keys, kind="stable"); ks = keys[order]
    starts = np.flatnonzero(np.r_[True, ks[1:] != ks[:-1]])
    return [(ks[a], order[a:b]) for a, b in zip(starts, np.r_[starts[1:], len(ks)])]
def _weighted(s, d, k, damp=0.0):
    kk = min(k, s.shape[1])
    top = np.argpartition(-s, kk - 1, axis=1)[:, :kk]
    st = np.take_along_axis(s, top, 1); dt = np.take_along_axis(d, top, 1)
    st = np.where(st > 0, st, 0.0)
    w = st.sum(1)
    return np.where(w > 0, (st * dt).sum(1) / np.where(w > 0, w + damp, 1.0), 0.0), (st >
        0).sum(1)
```

- `np.argsort(keys, kind="stable")` — positions that would sort the keys; 'stable' keeps equal keys in their original order.
- `np.r_[True, ks[1:] != ks[:-1]]` — True wherever a new key starts in the sorted list.
- `np.argpartition(-s, kk - 1, axis=1)[:, :kk]` — the kk largest values per row, without sorting the whole row (faster than argsort).
- `np.take_along_axis(s, top, 1)` — picks, row by row, the entries whose column positions are in top.

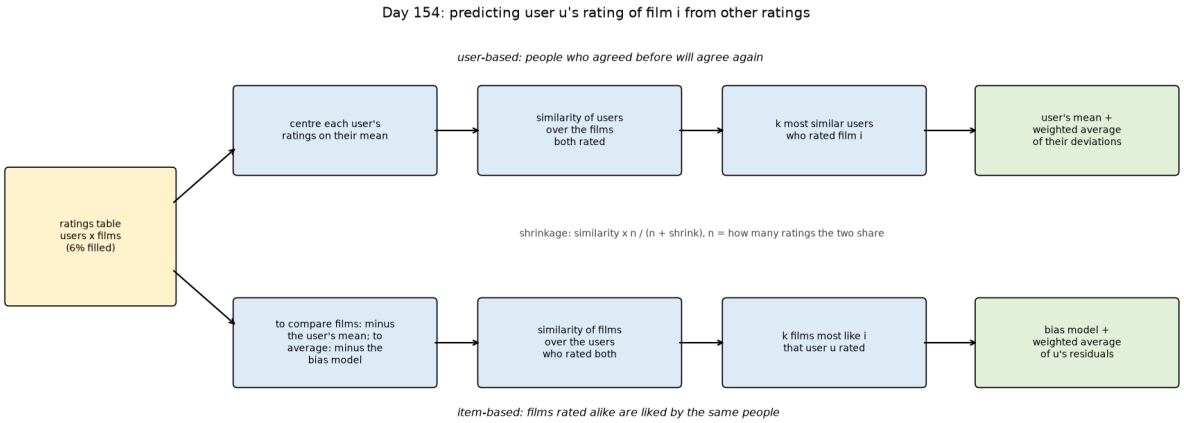
predict_user_knn and predict_item_knn

CODE

```
def predict_user_knn(D, M, S, users, items, k):
    out = np.zeros(len(users)); used = np.zeros(len(users), int)
    for ii, q in _groups(items):
        raters = np.flatnonzero(M[:, ii])
        if len(raters) == 0:
            continue
        s = S[np.ix_(users[q], raters)]
        d = np.broadcast_to(D[raters, ii], s.shape)
        out[q], used[q] = _weighted(s, d, k)
    return out, used
def predict_item_knn(D, M, S, users, items, k, damp=0.0):
    out = np.zeros(len(users)); used = np.zeros(len(users), int)
    for uu, q in _groups(users):
        rated = np.flatnonzero(M[uu])
        s = S[np.ix_(items[q], rated)]
        d = np.broadcast_to(D[uu, rated], s.shape)
        out[q], used[q] = _weighted(s, d, k, damp)
    return out, used
```

- `for ii, q in _groups(items):` — unpacks each (key, positions) pair; one film at a time for user-based.
- `S[np.ix_(users[q], raters)]` — np.ix_ builds a rectangular selection: these rows by these columns.
- `np.broadcast_to(D[raters, ii], s.shape)` — repeats one row of values for every asking user without copying memory.
- `out[q], used[q] = _weighted(...)` — two results assigned at once into the positions q.

SYSTEM DIAGRAM



User-based and item-based collaborative filtering.

Segment 2 — The experiments

Helpers: choosing the bias penalties, the control, the four similarities

CODE

```
def choose_biases(R, M, u, i, r):
    best = None
    for lu in LAMS:
        for li in LAMS:
            mu, bu, bi = fit_biases(R, M, lu, li)
            e = rmse(np.clip(mu + bu[u] + bi[i], 1, 5), r)
            if best is None or e < best[0]:
                best = (e, lu, li)
    return best
def shuffled(S, seed):
    n = S.shape[0]; iu = np.triu_indices(n, 1)
    v = S[iu][np.random.RandomState(seed).permutation(len(iu[0]))]
    T = np.zeros_like(S); T[iu] = v; return T + T.T
def item_parts(R, M, lam_u, lam_i, centre):
    mu, bu, bi = fit_biases(R, M, lam_u, lam_i)
    base = mu + bu[:, None] + bi[None, :]
    C = {"raw ratings (cosine)": 0.0, "minus the film's mean (Pearson)": item_means(R, M,
        mu)[None, :],
         "minus the user's mean (adjusted cosine)": user_means(R, M)[:, None], "minus the bias
             model": base}[centre]
    return (R - C) * M, (R - base) * M, base
```

- `if best is None or e < best[0]:` — the first result is always kept; later ones only if better.
- `np.triu_indices(n, 1)` — row and column positions of every cell above the diagonal: each pair once.
- `T + T.T` — copies the upper triangle into the lower one, so the shuffled matrix stays symmetric.
- `{...}[centre]` — a dictionary built and indexed in one expression: picks what to subtract by its name.

grid and demo_item: choosing on validation

CODE

```
def grid(R, M, D, S0, n, offset, u, i, r, predict, k_list, shrinks):
    E = np.zeros((len(shrinks), len(k_list)))
    for a, lam in enumerate(shrinks):
        S = S0 * n / (n + lam) if lam else S0
        for b, k in enumerate(k_list):
            c, _ = predict(D, M, S, u, i, k)
            E[a, b] = rmse(np.clip(offset + c, 1, 5), r)
    a, b = np.unravel_index(np.argmin(E), E.shape)
    return E, shrinks[a], k_list[b]
centre = min(best, key=lambda c: best[c][0]); _, lam, k, E = best[centre]
```

- `S0 * n / (n + lam) if lam else S0` — a conditional expression: shrink only when lam is not 0.
- `np.unravel_index(np.argmin(E), E.shape)` — argmin counts through the flattened table; this turns that count back into (row, column).
- `min(best, key=lambda c: best[c][0])` — the dictionary key whose stored validation error is smallest.

precision_at_10: scoring top-10 lists

MATH

precision@10 = hits / 10; recall@10 = hits / (liked held-out films)

CODE

```
def precision_at_10(Q, liked):
    who = np.flatnonzero(liked.sum(1) > 0)
    top = np.argsort(-Q, axis=1, kind="stable")[:, :10]
    h = np.take_along_axis(liked, top, 1)[who]
    return (h.sum(1) / 10).mean(), (h.sum(1) / liked[who].sum(1)).mean(), top[who], len(who)
```

- `np.argsort(-Q, axis=1, kind="stable")[:, :10]` — sorting the negated scores puts the highest first; the first 10 columns per row.
- `np.take_along_axis(liked, top, 1)[who]` — True where a recommended film is a liked test film, for the users who have any.
- `-np.inf` — rated films get minus infinity, so they always sort last and are never recommended.

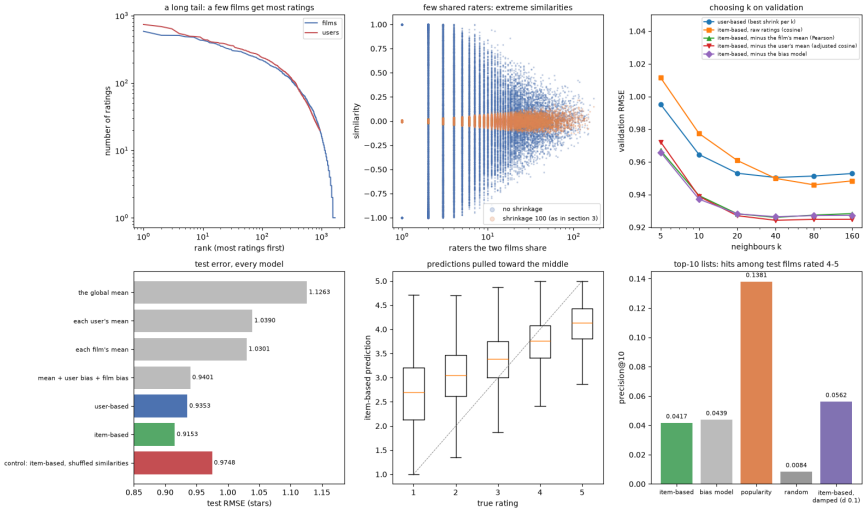
demo_topn: the damping search

CODE

```
for dmp in DAMPS:
    pv = precision_at_10(item_scores(Rf, Mf, lu, li, setting, dmp)[0], lv)[0]
    print(f"  d {dmp:<5} validation precision@10 {pv:.4f}")
    if best is None or pv > best[0]:
        best = (pv, dmp)
Pd = item_scores(R, M, lu, li, setting, best[1])[0]
```

- `item_scores(...)[0]` — the function returns four things; [0] keeps the first (the scores).
- `best = (pv, dmp)` — a tuple remembers the score and the setting together.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `urllib.request.urlretrieve(url, path)` — download a file to disk.
- `zipfile.ZipFile(z).extractall(folder)` — unpack an archive.
- `np.loadtxt(path, dtype=np.int64)` — read a table of numbers from a text file.
- `R[u, i] = r` — fill many cells at once by fancy indexing.
- `np.divide(a, b, out=..., where=...)` — safe division.
- `np.argpartition(-s, k - 1, axis=1)` — the k largest per row, unsorted.
- `np.take_along_axis(a, idx, 1)` — pick entries row by row.
- `np.ix_(rows, cols)` — a rectangular selection.
- `np.broadcast_to(v, shape)` — repeat without copying.
- `np.triu_indices(n, 1)` — each pair of n items once.

End of Day 154 syntax reference. For the experiments and what they showed, see the Day 154 complete guide.